MidEng GK8.2 Document Oriented Middleware using MongoDB

Grundlagen:

Als erstes habe ich meine Code angepasst + 2 neue Klassen geschrieben. EInmal WarehouseController und ProductController.

```java
package warehouse.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import warehouse.model.ProductData;
import warehouse.repository.WarehouseRepository;

import java.util.List;

@RestController  new *
@RequestMapping("/warehouse")
public class WarehouseController {

    @Autowired
    private WarehouseRepository repository;

    // 🔥 GET: alle Lager
    @GetMapping  new *
    public List<ProductData> getAllWarehouses() { return repository.findAll(); }

    // 🔥 GET: Lager nach ID
    @GetMapping("/{id}")  new *
    public List<ProductData> getWarehouseById(@PathVariable String id) { return repository.findByWarehouseID(id); }

    // 🔥 DELETE: Lager löschen
    @DeleteMapping("/{id}")  new *
    public void deleteWarehouse(@PathVariable String id) {...}
}
```

```java
package warehouse.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import warehouse.model.ProductData;
import warehouse.repository.WarehouseRepository;

import java.util.List;

@RestController  new *
@RequestMapping("/product")
public class ProductController {

    @Autowired
    private WarehouseRepository repository;

    // 🔥 POST: neues Produkt
    @PostMapping  new *
    public ProductData createProduct(@RequestBody ProductData product) { return repository.save(product); }

    // 🔥 GET: alle Produkte
    @GetMapping  new *
    public List<ProductData> getAllProducts() { return repository.findAll(); }

    // 🔥 GET: Produkt nach ID
    @GetMapping("/{id}")  new *
    public List<ProductData> getProductByProductID(@PathVariable String id) { return repository.findByProductID(id); }

    // 🔥 DELETE: Produkt löschen
    @DeleteMapping("/{id}")  new *
    public void deleteProduct(@PathVariable String id) {...}
}
```

Danach habe ich den Server gestartet mit gradle bootrun und anschließend getestet.

Products anzeigen:

localhost:8080/product

Quelltextformatierung

[{"warehouseID":"1","productID":"00-443175","productName":"Bio Orangensaft Sonne","productCategory":"Getraenk","productQuantity":2500.0,"id":"69b910037bd4680fef46ba79"},{"warehouseID":"1","productID":"00-871895","productName":"Bi
Apfelsaft Gold","productCategory":"Getraenk","productQuantity":3420.0,"id":"69b910037bd4680fef46ba7a"},{"warehouseID":"1","productID":"01-926885","productName":"Ariel Waschmittel
Color","productCategory":"Waschmittel","productQuantity":478.0,"id":"69b910037bd4680fef46ba7b"},{"warehouseID":"1","productID":"02-234811","productName":"Mampfi Katzenfutter
Rind","productCategory":"Tierfutter","productQuantity":1324.0,"id":"69b910037bd4680fef46ba7c"},{"warehouseID":"2","productID":"03-893173","productName":"Saugstauberbeutel
Ingres","productCategory":"Reinigung","productQuantity":7390.0,"id":"69b910037bd4680fef46ba7d"}]

Produkte von Warehouse 1:

localhost:8080/warehouse/1

Quelltextformatierung

[{"warehouseID":"1","productID":"00-443175","productName":"Bio Orangensaft Sonne","productCategory":"Getraenk","productQuantity":2500.0,"id":"69b910037bd4680fef46ba79"},{"warehouseID":"1","productID":"00-871895","productName":"Bio
Apfelsaft Gold","productCategory":"Getraenk","productQuantity":3420.0,"id":"69b910037bd4680fef46ba7a"},{"warehouseID":"1","productID":"01-926885","productName":"Ariel Waschmittel
Color","productCategory":"Waschmittel","productQuantity":478.0,"id":"69b910037bd4680fef46ba7b"},{"warehouseID":"1","productID":"02-234811","productName":"Mampfi Katzenfutter
Rind","productCategory":"Tierfutter","productQuantity":1324.0,"id":"69b910037bd4680fef46ba7c"}]

Produkt nach ID:

localhost:8080/product/00-871895

Quelltextformatierung

[{"warehouseID":"1","productID":"00-871895","productName":"Bio Apfelsaft Gold","productCategory":"Getraenk","productQuantity":3420.0,"id":"69b910037bd4680fef46ba7a"}]

eigenes Produkt hinzugefügt:

Ingres","productCategory":"Reinigung","productQuantity":7390.0,"id":"69b910037bd4680fef46ba7d"},{"warehouseID":"3","productID":"99-999999","productName":"Test
Produkt","productCategory":"Test","productQuantity":100.0,"id":"69b910c17bd4680fef46ba7e"}]

MongoDB prüfen:

```
luca_@MacBook-Air-4 DEZSYS_GK_WAREHOUSE_DOM % docker exec -it mongo mongosh
Current Mongosh Log ID: 69b941621d9625f58d9c28ed
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mon
Using MongoDB:          8.2.5
Using Mongosh:          2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mong
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-17T07:20:21.495+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage eng
   2026-03-17T07:20:21.530+00:00: Access control is not enabled for the database. Read and write access to data an
   2026-03-17T07:20:21.530+00:00: For customers running the current memory allocator, we suggest changing the cont
   2026-03-17T07:20:21.530+00:00: We suggest setting the contents of sysfsFile to 0.
   2026-03-17T07:20:21.530+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance probl
------

test> use test
already on db test
test> db.productData.find().pretty()
```

```
    _id: ObjectId('69b910c17bd4680fef46ba7e'),
    warehouseID: '3',
    productID: '99-999999',
    productName: 'Test Produkt',
    productCategory: 'Test',
    productQuantity: 100,
    _class: 'warehouse.model.ProductData'
  }
]
test>
```

EK + Vertiefung:

Als ertses habe ich mein Warehouse Repository angepasst, ProductController und Application:

```java
public interface WarehouseRepository extends MongoRepository<ProductData, String> {  6 usages  Thomas Micheler *

    public List<ProductData> findByProductID(String productID);  2 usages  new *
    public List<ProductData> findByWarehouseID(String warehouseID);  2 usages  Thomas Micheler

    List<ProductData> findByProductCategory(String category);  1 usage  new *

    List<ProductData> findByProductQuantityLessThan(double quantity);  1 usage  new *

    List<ProductData> findByWarehouseIDAndProductQuantityLessThan(String warehouseID, double quantity);  no usages  new *

}
```

```java
    // 🔥 Produkte nach Kategorie
    @GetMapping("/category/{category}")  new *
    public List<ProductData> getByCategory(@PathVariable String category) {...}

    // 🔥 Produkte unter bestimmtem Lagerbestand
    @GetMapping("/lowstock/{quantity}")  new *
    public List<ProductData> getLowStock(@PathVariable double quantity) {...}
}
```

MongoDB Queries:

1. Lagerbestand eines Produkts über alle Lager

```mongo
test> db.productData.find({ productID: "P-10" })
[
  {
    _id: ObjectId('69b944274b5d601bf5fbce17'),
    warehouseID: '1',
    productID: 'P-10',
    productName: 'Produkt-10',
    productCategory: 'Getraenk',
    productQuantity: 63,
    _class: 'warehouse.model.ProductData'
  }
]
```

2. Produkte mit weniger als 10 Stück

```
test> db.productData.find({ productQuantity: { $lt: 10 } })
[
  {
    _id: ObjectId('69b944274b5d601bf5fbceb8'),
    warehouseID: '4',
    productID: 'P-171',
    productName: 'Produkt-171',
    productCategory: 'Lebensmittel',
    productQuantity: 0,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbcee7'),
    warehouseID: '4',
    productID: 'P-218',
    productName: 'Produkt-218',
    productCategory: 'Waschmittel',
    productQuantity: 5,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbceef'),
    warehouseID: '5',
    productID: 'P-226',
    productName: 'Produkt-226',
    productCategory: 'Tierfutter',
    productQuantity: 1,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbcf24'),
    warehouseID: '4',
    productID: 'P-279',
    productName: 'Produkt-279',
    productCategory: 'Lebensmittel',
    productQuantity: 0,
```

3. Produkte nach Kategorie

```
test> db.productData.find({ productCategory: "Getraenk" })
[
  {
    _id: ObjectId('69b944274b5d601bf5fbce10'),
    warehouseID: '3',
    productID: 'P-3',
    productName: 'Produkt-3',
    productCategory: 'Getraenk',
    productQuantity: 171,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbce11'),
    warehouseID: '4',
    productID: 'P-4',
    productName: 'Produkt-4',
    productCategory: 'Getraenk',
    productQuantity: 424,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbce17'),
    warehouseID: '1',
    productID: 'P-10',
    productName: 'Produkt-10',
    productCategory: 'Getraenk',
    productQuantity: 63,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbce19'),
    warehouseID: '3',
    productID: 'P-12',
    productName: 'Produkt-12',
```

4. Lagerbestand eines Produkts in einem bestimmten Lager

```
test> db.productData.find({
|   productID: "P-10",
|   warehouseID: "1"
| })
[
  {
    _id: ObjectId('69b944274b5d601bf5fbce17'),
    warehouseID: '1',
    productID: 'P-10',
    productName: 'Produkt-10',
    productCategory: 'Getraenk',
    productQuantity: 63,
    _class: 'warehouse.model.ProductData'
  }
]
```

5 Crud Befehle:

CREATE:

```
test> db.productData.insertOne({
|   warehouseID: "1",
|   productID: "TEST-1",
|   productName: "Test Produkt",
|   productCategory: "Test",
|   productQuantity: 50
| })
{
  acknowledged: true,
  insertedId: ObjectId('69b945c363431fbba19c28ee')
}
```

READ:

```
test> db.productData.find().pretty()
[
  {
    _id: ObjectId('69b944274b5d601bf5fbce0e'),
    warehouseID: '5',
    productID: 'P-1',
    productName: 'Produkt-1',
    productCategory: 'Reinigung',
    productQuantity: 41,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbce0f'),
    warehouseID: '4',
    productID: 'P-2',
    productName: 'Produkt-2',
    productCategory: 'Tierfutter',
    productQuantity: 138,
    _class: 'warehouse.model.ProductData'
  },
  {
    _id: ObjectId('69b944274b5d601bf5fbce10'),
    warehouseID: '3',
    productID: 'P-3',
```

UPDATE:

```
Type "it" for more
test> db.productData.updateOne(
|   { productID: "TEST-1" },
|   { $set: { productQuantity: 999 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

DELETE:

```
}
test> db.productData.deleteOne({ productID: "TEST-1" })
{ acknowledged: true, deletedCount: 1 }
test>
```

UPDATE mehrere:

```
test> db.productData.updateMany(
|   { productCategory: "Getraenk" },
|   { $inc: { productQuantity: 10 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 52,
  modifiedCount: 52,
  upsertedCount: 0
}
```

Fragestellungen:

1. Nennen Sie 4 Vorteile eines NoSQL Repository

- Hohe Skalierbarkeit (horizontal über mehrere Server)
- Flexible Datenstruktur (kein festes Schema notwendig)
- Hohe Performance bei großen Datenmengen
- Gute Eignung für verteilte Systeme

2. Nennen Sie 4 Nachteile eines NoSQL Repository

- Keine festen Strukturen → Daten können inkonsistent sein
- Weniger Unterstützung für komplexe Joins
- Schwächere Transaktionssicherheit (kein vollständiges ACID)
- Standardisierung fehlt im Vergleich zu SQL

3. Welche Schwierigkeiten ergeben sich bei der Zusammenführung der Daten?

- Daten sind oft redundant gespeichert
- Es gibt keine Joins wie in relationalen Datenbanken
- Daten müssen manuell zusammengeführt werden
- Konsistenzprobleme können auftreten

4. Welche Arten von NoSQL Datenbanken gibt es?

- Dokumentenorientierte Datenbanken
- Key-Value Stores
- Spaltenorientierte Datenbanken
- Graphdatenbanken

5. Nennen Sie einen Vertreter für jede Art

- Dokumentenorientiert: MongoDB
- Key-Value Store: Redis
- Spaltenorientiert: Cassandra
- Graphdatenbank: Neo4j

6. Beschreiben Sie CA, CP und AP

- CA (Consistency + Availability):
  Alle Knoten liefern konsistente Daten und sind verfügbar, jedoch nicht partitionstolerant.
- CP (Consistency + Partition Tolerance):
  Konsistente Daten trotz Netzwerkfehlern, jedoch kann die Verfügbarkeit eingeschränkt sein.
- AP (Availability + Partition Tolerance):
  System bleibt verfügbar, auch bei Netzwerkproblemen, jedoch kann die Konsistenz leiden.